

COMP 10066

Week 12 Lecture

Security Issues and Performance
Testing

Lecture Overview

- Identifying Threats
- What is considered valuable data / information
- Performance Testing

Identifying Threats

- Create a list of threats to the application/system of interest.
 - Create a team to analyze issues – DOMAIN experts on team
 - Identify valuable assets
 - Financial
 - Personal Information
 - Trade secrets
 - Review Authorization and Authentication
 - Examine data flow (within system/between systems)
 - Identify weak points
 - **These are potential threats**

Ranking Threats

- Items to consider when ranking threats?
 - Damage Potential – Financial Personal information leaks
 - How likely is it for the attack to be successful?
 - Exploiter expertise required
 - Can they be a novice or must they be an expert.
 - Hacking expertise/software can be purchased
 - Affected users – How many users could possibly be affected.
 - Discoverability – How easily can the weakness be uncovered.

Testing for Software Security

- Software security testing falls under which generally category of software testing
 - Test to Fail
- Which programming languages are most prone to attack:
 - C / C++ programming languages
 - Memory overwriting (Buffer Overflow)
 - HTML / Javascript / SQL
 - via Injection (typically through a Web page interface)
 - Search box
 - <http://www.unixwiz.net/techtips/sql-injection.html#miti>

Examples of reducing vulnerability

- How have operating systems adjusted to reduce these type of vulnerability?
 - NX-Bit (No-Execute) bit is placed on all data within a Windows OS
 - Intel refers to this as the (eXecute Disable)
 - AMD used the name Enhanced Virus Protection
 - ARM refers to the feature as XN (eXecute Never)
 - Firewalls protect against port scanning (to some degree)

Examples of reducing vulnerability

- How do modern programming languages reduce the vulnerability of the software?
 - .NET / Java are virtual machine languages that have memory protection built-in.
 - Array out-of-bounds exceptions protect against buffer overrun explicitly
 - Variables must be initialized.

Website Vulnerability Scanner

- Free tools exist to help identify vulnerabilities of a website
- OWASP (maintains a list of these tools (both free and commercial)
 - OWASP - Open Worldwide Application Security Project
- https://owasp.org/www-community/Vulnerability_Scanning_Tools

Common software security attacks

- Summary of White Paper
 - <http://msdn.microsoft.com/en-us/security/gg598975.aspx>
- Attacks fall into 4 classifications which we will cover on the next few slides
 - Attacking Software Dependencies
 - Breaking Security Through the User Interface
 - Attacking Design
 - Attacking Implementation

Attacking Software Dependencies

- Applications rely on aspects of an environment to operate correctly (OS, other components (.NET environment, java runtime))
- The environment can be manipulated by:
 - blocking access to system components
 - Does application terminate if a DLL can't load
 - manipulate access via the Windows registry
 - Can access be modified from outside of the application
 - forcing application to use corrupt files
 - Will the corrupt data force the program into an unforeseen state?
 - manipulating files that the application creates, reads or writes during execution.
 - forcing the application to run in low memory, low disk space, low network bandwidth situations.

Attacking the User Interface

- The result could be privilege escalation (a normal user acquiring administrative rights) or allowing secret information to be viewed by an unauthorized user.
- Three most common techniques to "hack" a UI:
 - Buffer Overflow
 - Overflows are by far the most notorious security problems in software. They occur when applications fail to properly constrain input length. Some buffer overflows don't present much of a security threat. Others, however, can allow hackers to take control of a system by sending a well-crafted string to the application. This second type is referred to by the industry as "exploitable," because parts of the string may get executed if they are interpreted as code.
 - Examine all common switches and options
 - Options for running an application via command line switches can sometimes provide untested forcing application to use corrupt files
 - Will the corrupt data force the program into an unforeseen state?
 - Explore escape characters, character sets, and commands
 - Using different combinations of commands, escape characters specific to an environment that may cause unintended behavior. An example of this is entering command line commands into a text box. There are occasions where command entry that is not understood is passed to the underlying OS.

Attacking Design

- Some design decisions in an application can lead to security flaws.
 - Special Accounts – Like a Master Key
 - Sometimes special/default accounts are hardcoded or used as default when an application starts. These accounts are occasionally not as well protected as accounts that have proper username/password encryption.
 - Special Access via Testing automation – Test modes
 - Some testing modes are built right into applications so that they can be flipped into test mode during Beta testing or even after deployment. These modes can by-pass access and may open up access to capability/access that is limited.
 - Port Scanning – Looking for Open Doors
 - Used to try to gain access to a system through any open TCP/IP ports on a system. This is more of a system attack than an attack on a piece of software, but is useful for determining system characteristics.
 - Repetitive command execution – Denial of Service .
 - Used to try to Deny service the operating environment by intentionally overloading the host operating system.
 - Testing default values
 - Leaving values blank through default or clearing fields can sometimes lead to program using uninitialized variables. Though a bad programming practice, this can lead to unstable programming conditions (C/C++ programming languages suffer from this problem).

Attacking Implementation

- These attacks focus on how the system has been implemented or programmed
 - Get between time of check and time of use
 - When data is entered into an application it is important to have it used immediately. If it is stored temporarily (temp files), it is possible to attack the application via the temp file. The application would need to verify the data a second time before final use.
 - Create files with same name as files protected with a higher classification
 - Applications are often delivered as multiple components (DLL's). DLL's are typically loaded from locations where a user has limited privilege (Windows System folder). Often however they are loaded using the Path. Malicious DLL's can replace expected DLL's. This is
 - Force all error messages – Can reveal details about implementation / users
 - Error messages are generally intended to inform a user about an invalid state. They need to be carefully scrutinized to ensure that the information does not reveal any design/implementation details of the application or data source.

Protecting System access

How bad is it?

- Adobe (2013)
<https://nakedsecurity.sophos.com/2013/11/04/anatomy-of-a-password-disaster-adobes-giant-sized-cryptographic-blunder/>
- Sony (2014)
http://gawker.com/sonys-top-secret-password-lists-have-names-like-master_-1666775151
- Facebook (2019)
<https://krebsonsecurity.com/2019/03/facebook-stored-hundreds-of-millions-of-user-passwords-in-plain-text-for-years/>
- Are we learning anything?

Testing Account / Password Strength

- Special accounts must be eliminated from an application before release.
- Two aspects of passwords that can be controlled by a software developer
 - password strength
 - minimum length, type of characters required,
 - Applications should ensure the following minimum
 - 8 characters min (some say 10), 1 upper, 1 lower, 1 digit, 1 special
 - How passwords are stored
 - **Only store hashed passwords (not encrypted).**
 - The hash is re-computed on each password entry and the hashes are compared.
 - The plain-text password is never stored.

Testing Password Strength

- Improving a standard HASH
 - Add SALT to the password.
 - SALT (additional characters) can be added to the front back (or interspersed) with the actual password.
 - SALT should be unique for each user.
 - Account creation time – must be stored with data.
 - Iterate the (password/and salt) through the HASH algorithm multiple times (1000 to 10000).
 - See <http://en.wikipedia.org/wiki/PBKDF2>
 - Implementations available in most programming environments.
 - Java - <https://docs.oracle.com/javase/7/docs/api/javax/crypto/spec/PBEKeySpec.html>

Testing for Software Security

- Logging activity
 - Operating system logs
 - All Access / Authentication
 - Scan for particular account unauthorized access.
 - Specifically monitor the Windows directory and system DLL's for manipulation – Most virus checkers do this
 - Use automation to reported unusually high levels of access
 - Application level – A log of access and data entry along with invalid data entry is useful.
 - Useful to add to all Web Applications

Testing for Software Security

- In an Application Log what are some of the items that should be logged
 - Data being accessed.
 - Sensitive data should not be included in logs.
 - Component of the application that is currently active
 - Has the side effect of being useful for finding/confirming bugs
- Be Careful!
 - Logs must ensure that personal/financial data as this could also be vulnerable to leaks!
 - Logs are not typically encrypted within many systems.

What do we do as developers?

- Continually ask the questions about our own applications.
 - Are there any technical weaknesses?
 - Can we reduce the value (to others) of the data we store in our own systems?
 - Ensure that the data we store is protected by:
 - Limiting full access
 - Minimize the access of admin accounts.

Performance Testing

- Determines a systems performance under a particular load
 - Is the system responsive?
 - Is the system stable?
- Can be done as a load test where you are dealing with concurrent users
- Can also be done as a stress test where you try to overload the system and see how it responds
 - Similar to break point testing – find the point at which the system breaks under a given load

Performance Testing Tools

- Again, like security vulnerability identification, there are many tools out there for performance testing:
- <https://www.practitest.com/resource-center/blog/top-performance-testing-tools/>